

My AI+Education Application Demo presentation

This document includes a demo video and presentation of my recent developed AI app: AI-Powered Math Exam Analysis. It takes a student’s exam results as input and outputs:

- 1. An analysis of their strengths and weaknesses across different knowledge modules.
- 2. Targeted practice questions addressing the student’ s knowledge gaps

Demo video on Github: [Link](#)

Backup demo video on Tencent Docs: [Link](#)

Presentation:

Exam Paper Analysis LLM Model Technical Documentation - Tian Xiaohong

Version: v1.0 | Core: Lightweight LLM Model Training and Cursor Integration

🎯 Core Work Achievements

- **Model Training:** Based on education domain data, trained a specialized LLM model focused on exam paper analysis and example question generation, with in-depth optimization for educational scenarios
- **Private Cloud Deployment:** Deployed the trained model in a private cloud environment to ensure data security and stable, controllable model performance
- **Cursor Integration:** Connected the private cloud model to the Cursor editor via API interface to achieve a seamless workflow for exam paper analysis and exercise generation

1

Model Training

Trained a specialized LLM model focused on exam paper analysis and example question generation based on education domain datasets

2

Private Cloud Deployment

Deployed the trained model on private cloud servers to ensure data security and stable performance

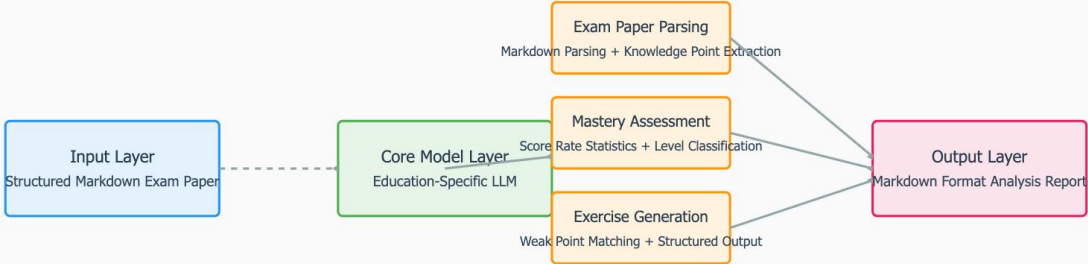
3

Cursor Integration

Integrated the model into the Cursor editor via API interface to enable seamless AI-assisted workflow

1. Core Functional Modules

1.1 Functional Architecture



2. Education Domain LLM Model Training and Private Cloud Deployment

2.1 Model Training

This project **independently trained** an LLM model specialized for the education domain, with in-depth optimization specifically for exam paper analysis and example question generation tasks. Core features of model training:

- **Domain-Specific:** Trained on large-scale education domain datasets, the model has a deep understanding of educational scenario characteristics such as exam paper structure, knowledge point systems, and scoring standards
- **Task-Focused:** Specifically optimized for core capabilities including exam paper parsing, knowledge module matching, subjective question scoring, weak point analysis, and example question generation
- **Mature Data:** Utilized a large amount of mature online exam question data and student answer data to ensure the model learns high-quality education domain knowledge

2.2 Private Cloud Deployment

Deployment Architecture: The trained education-specific LLM model is deployed on **private cloud servers** with the following advantages:

- **Data Security:** Private cloud deployment ensures all exam paper data is processed in a private environment without passing through third-party public clouds, protecting educational data privacy
- **Controllable Performance:** Computing resources in the private cloud environment can be configured according to actual needs to ensure model inference speed and stability

3. Core Technical Implementation Logic

3.1 Exam Paper Parsing Logic

The model achieves structured Markdown parsing through Prompt instructions, with the following core logic:

```
// Core parsing process (pseudocode) function parseMarkdownExam(examMarkdown) { // 1. Extract structured fields const questions = extractQuestions(examMarkdown, [ 'Question ID', 'Question Type', 'Score Value', 'Question Stem', 'Student Answer', 'Reference Answer' ]); // 2. Extract knowledge points and match modules questions.forEach(question => { const coreKnowledge = llm.extractCoreKnowledge(question.stem); question.knowledgeModules = llm.matchKnowledgeModules(coreKnowledge, knowledgeTree); }); // 3. Automatic scoring questions.forEach(question => { if (question.type === 'Objective Question') { question.score = question.studentAnswer === question.referenceAnswer ? question.fullScore : 0; } else { // LLM scoring for subjective questions const scoreResult = llm.gradeSubjectiveAnswer( question.stem, question.studentAnswer, question.referenceAnswer, question.fullScore ); question.score = scoreResult.score; question.deductionReason = scoreResult.reason; } }); return questions; }
```

3.2 Mastery Assessment Logic

Quantitative assessment of knowledge module mastery based on parsed question data:

```
// Mastery assessment process (pseudocode) function evaluateMastery(questions) { // 1. Group statistics by knowledge module const moduleStats = groupByKnowledgeModule(questions); // 2. Calculate score rate and level moduleStats.forEach(module => { module.totalScore = sum(module.questions.map(q => q.fullScore)); module.actualScore = sum(module.questions.map(q => q.score)); module.scoreRate = (module.actualScore / module.totalScore * 100).toFixed(1) + '%'; // Level classification if (module.scoreRate >= 85) module.masteryLevel = 'Excellent'; else if (module.scoreRate >= 70) module.masteryLevel = 'Good'; else if (module.scoreRate >= 50) module.masteryLevel = 'Weak'; else module.masteryLevel = 'Very Poor'; }); // 3. In-depth weak point analysis const weakModules = moduleStats.filter(m => m.masteryLevel === 'Weak' || m.masteryLevel === 'Very Poor'); weakModules.forEach(module => { module.weakPointAnalysis = llm.analyzeWeakPoints(module.questions); }); return { moduleStats, weakModules }; }
```

3.3 Exercise Generation Logic

Generate personalized exercises based on weak modules, with core logic including knowledge point matching, difficulty control, and format standardization:

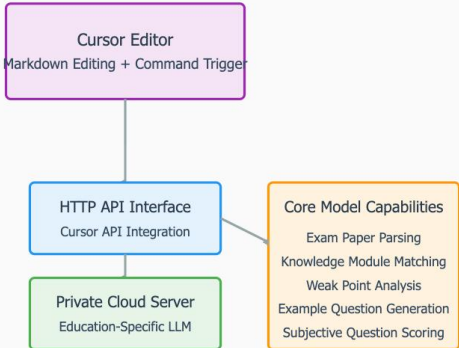
```
// Exercise generation process (pseudocode) function generatePracticeQuestions(weakModules, miniQuestionBank) { const practiceQuestions = []; weakModules.forEach(module => { // Extract core knowledge points const corePoints = module.weakPointAnalysis.map(analysis => analysis.corePoint); // Filter/generate questions by difficulty ratio const basicCount = Math.ceil(corePoints.length * 0.6); const advancedCount = Math.ceil(corePoints.length * 0.3); const extendCount = Math.ceil(corePoints.length * 0.1); // Basic questions (direct filtering) const basicQuestions = miniQuestionBank.filter(q => corePoints.includes(q.corePoint) && q.difficulty === 'Basic' ).slice(0, basicCount); // Advanced questions (variant generation) const advancedQuestions = corePoints.slice(0, advancedCount).map(point => llm.generateVariantQuestion(point, 'Advanced', miniQuestionBank) ); // Extended questions (cross-module comprehensive) const extendQuestions = corePoints.slice(0, extendCount).map(point => llm.generateComprehensiveQuestion(point, module.knowledgeModule, miniQuestionBank) ); // Merge and supplement analysis const moduleQuestions = [...basicQuestions, ...advancedQuestions, ...extendQuestions].map(q => ({ ...q, analysis: llm.generateQuestionAnalysis(q.stem, q.answer), targetWeakPoint: point })); practiceQuestions.push({ module: module.knowledgeModule, questions: moduleQuestions }); }); return practiceQuestions; }
```

4. Cursor Editor Integration Implementation

Core Achievement: Successfully integrated the education-specific LLM model deployed on private cloud into the Cursor editor, enabling a seamless AI-assisted workflow for exam paper analysis and exercise generation.

4.1 Model Integration Architecture

Connected the education-specific LLM model deployed on private cloud to the Cursor editor via **API interface** to enable direct invocation of model capabilities:



4.2 Integration Implementation Process

Integration Steps:

- API Configuration:** Configure the API interface address and authentication information of the private cloud model in the Cursor editor
- Model Registration:** Register the private cloud model as a custom model in Cursor, supporting access through Cursor's model invocation interface
- Command Design:** Design standardized command formats that allow users to trigger model functions through natural language instructions
- Result Display:** Model analysis results are directly inserted into the Cursor editor in Markdown format, supporting real-time editing and export

Integration Advantages: By integrating the specialized LLM model into Cursor, an AI-assisted workflow for educational scenarios has been achieved. The model can be directly invoked within Cursor without switching tools, improving the efficiency of exam paper analysis and exercise generation. The command design adopts a "fixed prefix + parameter configuration" mode to facilitate model parsing of user requirements; the output format strictly follows Markdown specifications to ensure direct editing, copying, and exporting within Cursor.